

TPFI 2024/25

Hw 2: Efficienza, Tipi & Prove

docente: I. SALVO, Sapienza Università di Roma
assegnato: 3 aprile 2026, consegna 20 aprile 2026

1. mergeSort “iterativo”

1. Definire una funzione Haskell che segue la seguente idea *bottom-up* (in un qualche senso “iterativa”) per implementare l’algoritmo *mergeSort*:

Data una lista xs , creare una lista di liste lunghe 1, ciascuna contenente un elemento di xs , poi fondere a due a due le liste ordinate (eventualmente lasciando inalterata l’ultima lista quando il numero delle liste è dispari), finchè non rimane un’unica lista ordinata.

Ad esempio, cominciando con $[5,3,4,2,1]$ la funzione calcola le seguenti liste: $[[5],[3],[4],[2],[1]]$, poi $[[3,5],[2,4],[1]]$, $[[2,3,4,5],[1]]$ e infine $[[1,2,3,4,5]]$ da cui viene estratto il risultato finale $[1,2,3,4,5]$.

2. Accelerare la prima fase di questo algoritmo per trarre vantaggio da input “favorevoli”. La migliorìa dovrebbe assicurare un comportamento lineare in casi particolarmente fortunati e più in generale una complessità $\Theta(n \log k)$ dove k è... (completare la frase).

2. Alberi & funzionali sugli alberi

Considerare le seguenti definizioni di alberi binari:

```
data BinTree a = Node a (BinTree a) (BinTree a) | Empty
data BinTree' a = Node' (BinTree' a) (BinTree' a) | Leaf a
```

1. Scrivere i funzionali *mapBT*, *mapBT'* che generalizzano agli alberi *BinTree* e *BinTree'* l’analogo funzionali *map* sulle liste;

2. Scrivere i funzionali *pRecBT*, *pRecBT'* che definiscono la ricorsione primitiva sugli alberi binari (sulle liste la ricorsione primitiva è implementata dal funzionale *foldr*, ma in questo caso attenti a non “linearizzare” le computazioni come fanno le funzioni di famiglia *fold* sui tipi di classe *Foldable*).

Riflettete accuratamente sui tipi che devono avere e su quali siano, di fatto, i principi di ricorsione sugli alberi binari.

3. Scrivere poi le seguenti funzioni usando *pRecBT* e *pRecBT'* (cercare di ottenere algoritmi lineari nel numero dei nodi):

- (a) numero dei nodi di un albero binario;
- (b) altezza dell'albero (= lunghezza in numero di archi del più lungo cammino radice-foglia)
- (c) massimo indice di sbilanciamento (= massima differenza tra altezza sotto-albero destro/sinistro)

FACOLTATIVO: Gli alberi a branching illimitato si possono facilmente definire in Haskell come segue: `data Tree a = K a [Tree a]`. Come ai punti precedenti, scrivendo i funzionali `mapT` e `pRecT`.

3. Minimo Antenato comune

Considerando la definizione `BinTree` dell'esercizio precedente (con chiavi anche nei nodi interni), dato un albero B e due valori u e v (assumete per semplicità le chiavi uniche in B), il *minimo antenato comune* di u e v (**lca**(u, v) per *low common ancestor*) è il più piccolo sottoalbero di B che contiene sia u che v .

Scrivere una funzione `lca :: BinTree a → a → a → BinTree a` seguendo due possibile vie:

1. scrivere e poi usare una funzione `pathToX :: BinTree a → a → [a]`, tale che `pathToX B x` restituisce la lista contenente la sequenza di chiavi incontrata nel cammino che lega la radice di B al nodo con chiave x ;
2. facendo un'unica visita di B ma senza usare strutture dati ausiliarie di dimensione dipendente dalla misura dell'albero (altezza o numero di nodi).
3. migliorare la complessità delle due funzioni precedenti nel caso valga la precondizione che B sia un albero binario di ricerca;
4. FACOLTATIVO: dare una soluzione nel caso in cui l'albero sia a branching illimitato (il tipo `Tree` dell'esercizio facoltativo precedente).

4. Derivazioni di programmi

La funzione `scanr :: (a → b) → b → [a] → [b]` può essere facilmente definita componendo `map`, `foldr` e `tails` (chiamata *suffissi* nell'Homework precedente):

```
scanr f e = map (foldr f e) . tails
```

Usare la definizione sopra come specifica per derivare una definizione efficiente (cioè lineare nella lunghezza della lista) facendo manipolazioni algebriche, in analogia con quanto visto per `scanl` (slide lezione **9bis**).